

COAL LAB-11

TASK # 3

Objective:

Reverse the order of words in a string using a stack in EMU8086.

- Input: HELLO WORLD
- Output: WORLD HELLO
- Letters inside each word remain in correct order.

Memory Data:

Code:

```
.model small
```

```
.stack 100h
```

```
.data
```

```
str db 'HELLO WORLD$.'
```

Explanation:

- Small model → single code and data segments.
- Stack size = 256 bytes.
- \$ is used as end-of-string marker.

Step-by-Step Code and Explanation:

1. Initialize Data Segment and Pointer:

Code:

```
mov ax, @data
```

```
mov ds, ax
```

```
mov si, offset str
```

Explanation:

- Load data segment into DS.
- SI points to the first character of the string.

2. Find the End of the String:

Code:

find_end:

```
mov al, [si]
cmp al, '$.'
je start_reverse
inc si
jmp find_end
```

Explanation:

- Move forward until \$ is found.
- After the loop, SI points at \$.

Code:

start_reverse:

```
dec si
```

Explanation:

- Move one step back.
- Now SI points to the last character 'D'.

3. Start Reverse Processing Loop:

Code:

reverse_loop:

```
cmp si, offset str
jb done
```

Explanation:

- Continue while SI is within the string.
- Stop when SI goes before the start.

4. Count Characters of Current Word (Backward):

Code:

```
mov cx, 0
count_word:
    cmp si, offset str
    jb push_word
    mov al, [si]
    cmp al, ' '
    je push_word
    inc cx
    dec si
    jmp count_word
```

Explanation:

- Move backward from the end.
- Stop at the space or the start of the string.
- CX stores the word length.

For WORLD → CX = 5.

5. Set Pointer to Start of the Word:

Code:

```
mov bx, cx
mov di, si
inc di
```

Explanation:

- SI is in space before the word.
- DI = SI + 1 → first letter of the word

6. Push Word Characters onto Stack:

Code:

```
push_chars:
    cmp bx, 0
    je print_word
    mov al, [di]
    mov ah, 0
    push ax
    inc di
    dec bx
    jmp push_chars
```

Explanation:

- Push letters of the word onto the stack.
- For WORLD: push W O R L D.

7. Pop and Print Characters:

Code:

```
print_word:
    mov bx, cx
print_chars:
    cmp bx, 0
    je print_space
    pop dx
    mov ah, 2
    int 21h
    dec bx
    jmp print_chars
```

Explanation:

- Stack reverses order once.
- Popping reverses again.
- Word appears in original letter order.

8. Print Space Between Words:**Code:**

```
print_space:  
    mov dl, '  
    mov ah, 2  
    int 21h
```

Explanation:

- Adds space after each word.

9. Move to Previous Word:**Code:**

```
dec si  
jmp reverse_loop
```

Explanation:

- Move left of the space.
- Process previous word (HELLO).

10. Program Termination:**Code:**

```
done:  
    mov ah, 4ch  
    int 21h
```

Explanation:

Exit to DOS.

11. Final Output

Input:

HELLO WORLD

Output:

WORLD HELLO

THANKYOU